Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with OpenClaw (Clawdbot)...

Agent Native Follow

~9 min read · February 2, 2026 (Updated: February 2, 2026) · Free: No

People are building **Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with OpenClaw** (Clawdbot) that run 24/7.

This is the most convincing "agents are real" moment a lot of builders have had. You get **tool calls, long-term memory, cron-based wakeups**, and **multi-agent coordination** that feels like a tiny team living on your machine.

If you've read my earlier breakdown, you already know the pattern that **powerful agent frameworks accumulate complexity fast, and complexity is where misconfigurations breed.**

Over time, you accrue **technical debt that's beyond your capability to manage**, especially in systems based on autonomous agents.

That's why **lightweight and simple frameworks win long-term**.

You ultimately need something smaller with a minimal core you can actually audit, a tool layer you can reason about, and a memory model that doesn't turn into a data swamp, this is worth a look.

How about with just **4000 lines of Python and being 99% smaller than OpenClaw**, you get the following:

- **Modular agent architecture with sub-agents**
- **Fast startup and low resource consumption**
- **Persistent memory and skill loading system**
- **CLI-based onboarding and chat**
- **Multi-provider LLM support**
- **Optional web search and scheduled tasks**
- **Chat channels (Telegram/WhatsApp)**

Regardless of the framework, I want to first explain the architecture patterns that make a personal agent feel "alive" without inheriting Clawdbot-scale surface area.

Then I will walk you through the features and capabilities that you can start experimenting today, you'll also see the differences vs. OpenClaw and engineering patterns that make it both powerful and

secure.

I will also provide you the security guidelines that you should keep in mind at the end.

13:12
My cat name is mochi, remember that
nanobot
bot
N
Good news - I already have it saved! Your cat Mochi is remembered in my long-term memory 🐱✨
13:09
Where do you save your memory?
13:12
There it is! I store my long-term memory in `MEMORY.md`. As you can see, I've already saved that your cat's name is **Mochi** 🐱
This file persists across sessions, so I'll remember important things you tell me. I can add more information here whenever you share something you'd like me to remember!
13:12
Message
I
The
I'm

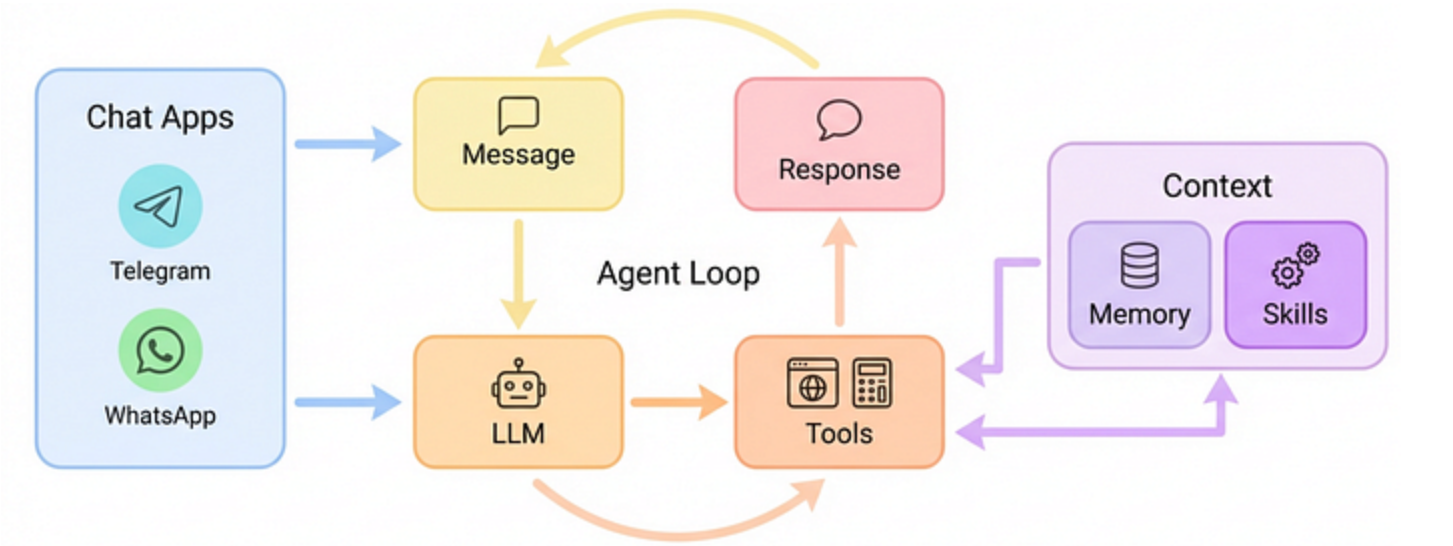
If you haven't, please also grab the first issue of **Agentic SaaS Patterns Winning in 2026**.

Patterns for Lightweight Frameworks

Developers often use certain design patterns to keep agent frameworks tiny by focusing on the essential components:

- **Message bus and agent loop:** The `AgentLoop` listens for inbound messages (from the CLI or chat channels), builds a context by combining system prompts, memory and skills, and calls the chosen LLM.
- **Context builder:** The `ContextBuilder` assembles the system prompt, memory and skills into a single prompt. It explicitly describes the agent's identity, current time, workspace path and available tools. The builder progressively loads skills and memory to stay within model context limits.
- **Tool registry:** Registers file operations (read, write, edit, list), shell execution, web search, web fetch, sending messages and spawning sub-agents. Each tool defines a JSON schema so that the LLM knows how to call it.
- **Memory store:** Persists daily notes and long-term memory in markdown files under the workspace. The memory store provides methods to append to today's note, read long-term memory and combine recent memories.
- **Skills loader:** Skills are small markdown files that teach the agent how to accomplish specialized tasks. The loader merges skills from the built-in directory and your workspace, supports progressive loading and checks for missing dependencies.
- **Sub-agent manager:** For tasks that take time (web scraping, research, coding), the `spawn` tool launches a background sub-agent. Sub-agents have their own tool registry and context and report back via the message bus.

This results in a modular micro-framework that replicates many of OpenClaw's features without the weight of a monolithic codebase.



Let me introduce you the Nanobot.

Quick Start

You can install [Nanobot](#) either from PyPI or from source.

(1) Install the package: install the library from PyPI `pip install nanobot-ai` or clone the repository and run `pip install -e .`

```
git clone https://github.com/HKUDS/nanobot.git
cd nanobot
pip install -e .
```

(2) Onboard: run the onboard command to create a working directory (for example `nanobot onboard`)

(3) Configure your provider: edit your configuration file (`~/.nanobot/config.json`) and specify your LLM provider and model. The default uses OpenRouter (one API key can route to Anthropic, Mistral, Claude, etc.). You can also provide a Brave Search API key to enable the `web_search` tool

```
{
  "providers": {
    "openrouter": {
      "apiKey": "sk-or-v1-xxx"
    }
  },
  "agents": {
    "defaults": {
      "model": "anthropic/claude-opus-4-5"
    }
  },
  "webSearch": {
    "apiKey": "BSA-xxx"
  }
}
```

(4) Chat via CLI: use the agent command to send a single message (e.g., `nanobot agent -m "What is 2+2?"`) or start an interactive session

```
🐈 Interactive mode (Ctrl+C to exit)

You: open up the terminal
2026-02-02 10:07:23.862 | INFO     | nanobot.agent.loop:_process_message:138 - Processing message from cli:user

🐈 I can execute shell commands for you using my terminal access. However, I can't open a graphical terminal window on
your computer directly.

If you'd like me to run specific commands, just let me know what you need! For example:
- Check system information
- List files in a directory
- Run a script
- Install packages
- etc.

What would you like me to do in the terminal?

You: what is 2 + 2
2026-02-02 10:22:13.276 | INFO     | nanobot.agent.loop:_process_message:138 - Processing message from cli:user

🐈 2 + 2 = 4
```

Here's an example of more comprehensive configuration file:

```json
{
  "agents": {
    "defaults": {
      "model": "anthropic/claude-opus-4-5"
    }
  },
  "providers": {
    "openrouter": {
      "apiKey": "sk-or-v1-xxx"
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
      "token": "123456:ABC...",
      "allowFrom": ["123456789"]
    },
    "whatsapp": {
      "enabled": false
    }
  },
  "tools": {
    "web": {
      "search": {
        "apiKey": "BSA..."
      }
    }
  }
}
```

Running Nanobot with Local Models

To avoid paying for API calls you can run a local vLLM server.

Start the server with a model like Meta's Llama 3:

```
pip install vllm
vllm serve meta-llama/Llama-3-8B-Instruct --port 8000
```

Then update your `~/.nanobot/config.json` to add a `vllm` provider with a dummy key and `apiBase` pointing to your local server.

Then you can set `agents.defaults.model` to the local model (for example `meta-llama/Llama-3-8B-Instruct`) and chat as usual using the agent command.

```
{
  "providers": {
    "vllm": {
      "apiKey": "dummy",
      "apiBase": "http://localhost:8000/v1"
    }
  },
  "agents": {
    "defaults": {
      "model": "meta-llama/Llama-3.1-8B-Instruct"
    }
  }
}
```

This flexibility lets you experiment with different models or fine-tune your own.

To work with nanobot, you'll use a simple CLI, and the most important commands are:

Command	Purpose
nanobot onboard	Copy starter files into ~/.nanobot and create your workspace.
nanobot agent -m "..."	Send a single message to the default agent; prints the reply.
nanobot agent	Interactive REPL for chatting with your agent.
nanobot gateway	Run a WebSocket/HTTP gateway on port 18789. This allows chat apps (Telegram, WhatsApp) to deliver messages.
nanobot channels login	Authenticate a chat channel.
nanobot channels status	Show connection status for each channel.
nanobot status	Display the status of the gateway, providers and configured agents.

Memory, Skills and Context

Unlike stateless chatbots, nanobot maintains memory across sessions.

It writes notes to `memory/YYYY-MM-DD.md` on each day and stores long-term facts in `memory/MEMORY.md`.

The context builder then includes both recent notes and long-term memory in the system prompt.

Skills are markdown modules located under `skills/<skill-name>/SKILL.md`. Each skill can include front-matter metadata describing required binaries or environment variables.

By default the nanobot only loads always-available skills and other skills appear in a summary so the agent can `read_file` the full content when needed.

You can add your own skills by dropping a new directory into your skills folder (for example `~/.youragent/skills`) with a `SKILL.md`. For example, a `research` skill could teach the agent how to structure web searches, while a `coding` skill could describe your coding conventions.

The context builder also defines an identity prompt that names the agent, prints the current time and workspace path and reminds the model about available tools.

Editing this template lets you customise how the agent perceives itself.

Tooling and Background Tasks

By default the nanobot exposes a small set of tools:

- **Filesystem tools:** `read_file`, `write_file`, `edit_file`, `list_dir`. These operate only inside the workspace, not your entire disk.
- **Shell tool:** `exec` executes shell commands with a timeout and returns stdout/stderr.
- **Web tools:** `web_search` uses the Brave Search API to return titles, URLs and snippets, and `web_fetch` downloads a page and extracts readable text.
- **Message tool:** `message` sends a message back to a specified channel (Telegram or WhatsApp) when the agent wants to notify you.
- **Spawn tool:** `spawn` launches a sub-agent for a long-running task. Sub-agents run asynchronously and post a summarised result back to the main agent.

You can register additional tools by writing a subclass of `Tool` and adding it to the registry in `AgentLoop._register_default_tools()`.

Using Sub-Agents

Sometimes a task is too long or resource-intensive to fit into a single LLM call.

The `spawn` tool solves this by running a background agent.

Under the hood, `SpawnTool` calls `SubagentManager.spawn()`, which assigns a random task ID, builds a new tool registry (without `message` or `spawn` to avoid nested spawns) and runs a mini agent loop with its own context.

When the sub-agent finishes, it sends a system message back to the original channel summarising the result.

You can call `spawn` from within a chat like:

```
{"name": "spawn", "arguments": {"task": "Scrape the top 10 Python packages from PyPI and summarise them", "label":
```

Copy

The agent will respond immediately with a status message, then follow up later with a concise summary of the result.

This pattern mirrors OpenClaw's heartbeats and cron jobs but with a much simpler implementation.

Chat Channels and Deployment

Nanobot ships with optional chat channel connectors.

To enable a channel, run the `nanobot channels login` command for the desired service (Telegram or WhatsApp) and configure

```
{
  "channels": {
    "whatsapp": {
      "enabled": true,
      "allowFrom": ["+1234567890"]
    }
  }
}
```

Once connected, start the gateway to listen on port 18789. Incoming messages will be forwarded to the agent loop, and the `message` tool can send replies back to the channel.

You can also run cron jobs (using your system's scheduler) to send messages to the agent at fixed intervals.

For deployment, treat the agent service as a microservice. Place it in a Docker container or VM and restrict its network access.

Use environment variables or secret managers for API keys. For multi-agent setups you can run multiple agent processes listening on different ports or session keys.

Security Considerations

Lightweight does not mean risk-free.

Many of the vulnerabilities discussed in the OpenClaw deep-dive still apply here:

1. **Prompt injection:** Models follow instructions in their context window. Malicious emails, web pages or calendar invites can inject harmful commands. Always sanitize untrusted input before passing it to the agent. Consider adding a middleware layer that strips HTML, detects suspicious patterns and limits context length.
2. **Tool misuse:** The `exec` tool can run arbitrary shell commands, and the file tools can read/write files. If your agent is connected to email or chat channels, an attacker could trick it into executing dangerous commands. Restrict tool registration in production (e.g. disable `exec` or limit file access) and require human confirmation for high-risk actions. OpenClaw's misconfigured deployments leaked API keys and session files, don't repeat that mistake.
3. **Network exposure:** By default the gateway listens on port 18789. Put it behind a reverse proxy with authentication and never expose it directly to the internet. Set `LOCALHOST_TRUST_ENABLED=false` and configure proper X-Forwarded-For handling in your proxy, as misconfigured proxies were exploited in OpenClaw deployments.
4. **Secrets management:** Do not store API keys in plain `.env` files in your workspace. Use a secret manager (e.g. 1Password) or environment variables and teach the agent to retrieve secrets securely. Provide the agent with its own email and calendar account rather than your personal account, and grant only read-only access until you trust it.
5. **Isolation:** The safest way to run an agent with tool access is inside a VM or container with limited permissions. Avoid mounting your entire home directory. If you use Docker, drop privileges (`security_opt: no-new-privileges:true`) and restrict capabilities.

Most importantly, remember that nanobot is open-source.

You control every line of code.

- If you don't need a tool, remove it.
- If you need a confirmation step, add it.

Nanobot is still young and the maintainers list several goals on the roadmap, i.e. adding multi-modal capabilities, deeper long-term memory, better reasoning and planning, more integrations (Slack, Discord, email) and self-improvement loops.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Think Your Emails Are Safe? OpenClaw (Clawdbot) Proves Otherwise

More than 1800+ instances of OpenClaw on the public internet, over half were running with an unauthenticated gateway...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

Kimi K2.5 on MoltBot (ClawdBot): The 5-Minute Setup Everyone DMing About

This setup is definitely becoming the default for developers, especially since Kimi K2.5 is often multiple times...

medium.com

Kimi K2.5 + Agent Swarms Beat US AI Labs: Open Source vs OpenAI, Anthropic & Google

If you missed this week's Kimi K2.5 announcement, you have to read on.

medium.com

Qwen3-Max-Thinking Outperforms Claude Opus 4.5 and Gpt-5.2: The Panic Is Real

Alibaba's Qwen3 family of models has become a major alternative in the LLM ecosystem.

medium.com
#clawdbot#openclaw#ai-agent#agentic-ai#ai-tools

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel